Lab 16 — Resources and operations

Theory — what a shared cluster demands from every application: say what it consumes, accept a budget, be observable and debuggable, hold only the rights it needs, and talk only to whom it must.

Objectives

- Declare **requests** and **limits**, understand the **QoS** classes, `OOMKilled` and `Pending`.
- Partition with **namespaces**, and bound with **LimitRange** and **ResourceQuota**.
- Observe and debug: `kubectl top`, `events`, `logs --previous`, `describe`, `kubectl debug`.
- Read and write a minimal **RBAC**: ServiceAccount, Role, RoleBinding, `kubectl auth can-i`.
- Restrict the network with a **NetworkPolicy**.

---

1. A cluster is shared

Until now your cluster had a single tenant: you. In a company, one cluster hosts dozens of applications and teams on the same nodes. Three questions become vital: how much does each Pod consume (to place and protect it), who is allowed to do what (so nobody deletes the neighbour's Deployment), and who may talk to whom (so a hole in the front end does not open the accounting database). Lab 10 asked those questions for one container; the cluster answers them with objects.

> **→ DOCKER RECAP**
> `--memory 256m --cpus 0.5` become `resources.limits`; the OOM killer and code `137` are the same (Pods run in the same cgroups, lab 10); `podman stats` becomes `kubectl top`; `podman logs --previous` ... did not exist, but `podman logs` on a container relaunched by `--restart` raised the same problem; and `--cap-drop` / `--read-only` come back unchanged in the Pod's `securityContext`.

2. Requests and limits: saying what you consume

```
resources:
  requests: { cpu: "100m", memory: "64Mi" }     # what the scheduler reserves
  limits:   { cpu: "500m", memory: "256Mi" }    # what the cgroup caps
```

The **request** is a **reservation**: the scheduler only places the Pod on a node where the sum of requests stays under capacity, and it is what `kubectl describe node` counts under `Allocated resources`. The **limit** is the **cgroup ceiling** of lab 10: memory beyond it kills the container (`OOMKilled`, code `137`, and the kubelet restarts it into `CrashLoopBackOff`), CPU beyond it is **slowed down** (*throttling*), never killed. `100m` = a tenth of a core; `64Mi` = 64 mebibytes.

QoS class	Condition	Fate under memory pressure on the node
`Guaranteed`	requests = limits for CPU and memory	evicted last
`Burstable`	requests set, limits different or partial	evicted after the BestEffort ones
`BestEffort`	no request and no limit	evicted first

The kubelet derives the class from the spec (`status.qosClass`); it decides who gets sacrificed when a node runs out of memory. A database in `BestEffort` is a database that will be killed first.

> **→ JAVA**
> The JVM sees the **limit** as total memory (lab 10): with `limits.memory: 256Mi` and `-XX:MaxRAMPercentage=75`, the heap is 192 Mi, and the rest (metaspace, threads) has to fit in the remaining 64 Mi. An `OOMKilled` on a Java API almost always comes from an over-ambitious percentage or a limit that is too tight. For CPU, a request of `100m` is enough to run, but a starting JVM eats a whole core for ten seconds: a CPU `limit` that is too low stretches start-up and makes the probes fail — many teams set **no** CPU limit at all.

A request that is too large has another effect: the Pod stays `Pending` with `FailedScheduling … 0/1 nodes are available: 1 Insufficient memory`. Nothing happens until a node can host it — the most common symptom after scaling up.

3. Namespaces, LimitRange, ResourceQuota

The **namespace** partitions names (two `api` in two namespaces), carries the RBAC rights and quotas, and is deleted as one block. The usual layout: one namespace per application and per environment (`billing-prod`, `billing-staging`), or one per team. `kube-system` holds the cluster's components; `default` should contain nothing.

Two objects bound a namespace. The **LimitRange** sets **default values** (a Pod created without `resources` receives `defaultRequest` and `default`) and per-container **ceilings** (`max`): a Pod asking for more is **rejected** at `apply` time (`maximum memory usage per Container is 1Gi, but limit is 4Gi`). The **ResourceQuota** sets a **total budget**: sum of requests, of limits, number of Pods, Services, PVCs — `kubectl describe quota` shows `Used` and `Hard`. With a quota in force, every Pod **must** declare resources (or have a LimitRange that supplies them).

> **K3S**
> A single node makes all of this visible at once: `kubectl describe node` shows the capacity of your WSL (what `.wslconfig` gave it) and the requests of every Pod, including those in `kube-system`. On a managed cluster, the platform team puts a LimitRange and a ResourceQuota in every namespace it hands you: a Pod without `resources` is rejected there or given defaults — which is why a company manifest always declares them.

4. Observing and debugging

Need	Command
real consumption (CPU, memory)	`kubectl top nodes`, `kubectl top pods` (metrics-server, shipped with k3s)
what happened	`kubectl get events --sort-by=.lastTimestamp`, `kubectl describe pod` (Events section)
why the previous container died	`kubectl logs pod --previous`
explore a container without a shell	`kubectl debug pod -it --image=alpine --target=container --profile=general`
copy a file	`kubectl cp pod:/path ./file`
everything about an object	`kubectl get pod -o yaml` (status, conditions, `lastState.terminated.reason`)

`kubectl debug` deserves a word: it adds an **ephemeral container** to the Pod, in the same process namespace as the target (`--target`), with the tools you want (`alpine`, `busybox`, `nicolaka/netshoot`). It is the answer to the distroless image of lab 05: no shell in the image, a shell next to it for the duration of the diagnosis. The ephemeral container disappears with the Pod and never enters the template.

The checklist for an incident is still that of lab 12: `kubectl get pods` (state, `RESTARTS`), `describe` (Events, `Last State`), `logs --previous`, `top`; then, for the network, the chain of lab 13.

5. RBAC: grant only what is needed

Every call to the API server is **authenticated** (certificate, token, OIDC) and then **authorised** by **RBAC** (*role-based access control*). Four objects:

Object	Scope	Content
Role / **ClusterRole**	one namespace / the whole cluster	rules: `resources` + `verbs` (`get`, `list`, `watch`, `create`, `update`, `patch`, `delete`)
RoleBinding / **ClusterRoleBinding**	one namespace / the whole cluster	ties a role to subjects: users, groups, **ServiceAccounts**

A **ServiceAccount** is an application's identity inside the cluster; every Pod has one (`default` if nothing is said), whose token is mounted under `/var/run/secrets/kubernetes.io/serviceaccount/`. An application that never calls the API needs no right at all — and `automountServiceAccountToken: false` even takes the token away. `kubectl auth can-i <verb> <resource> --as=system:serviceaccount:<ns>:<name>` tests a permission without logging in again, and `--list` enumerates everything a subject may do.

> **→ SECURITY**
> The built-in ClusterRoles (`view` , `edit` , `admin` , `cluster-admin`) cover most human needs: `view` to read without the Secrets, `edit` to deploy, `admin` to manage the namespace's RBAC. `cluster-admin` is the cluster's root: never for an application, rarely for a human. The k3s kubeconfig **is** `cluster-admin` — hence lab 11 and its `644` . The **least privilege** of lab 10 applies: a Pod that does not need the API has an empty ServiceAccount, and an operator that does has a Role listing exactly its resources.

6. NetworkPolicy: talk only to whom you must

By default, **every Pod can reach every Pod**, in all namespaces (lab 13). A **NetworkPolicy** selects Pods (`podSelector`) and declares what is allowed inbound (`ingress`) and/or outbound (`egress`); as soon as a policy selects a Pod, **everything not listed is refused**. The lab's example: only `app=web` Pods may reach `app=api` on `8080` ; a client without that label gets `Connection refused` . Policies add up (union of permissions), and common practice starts with a *deny-all* policy per namespace, then explicit allowances.

> **→ NETWORK**
> The NetworkPolicy is an API object; it is the **CNI** that enforces it (or not). Flannel alone does not; k3s ships a policy controller (kube-router) that programs filtering rules on every node, Calico and Cilium do the same with more finesse (layer 7, FQDN). On a cluster whose CNI ignores policies, the object is accepted and **has no effect**: check with a test, as in the lab. The rules are layer 3/4 (IP, port): the NetworkPolicy does not replace authentication between services.

7. In the company

- **Every manifest declares** `resources` , with measured requests (`kubectl top` under load) and memory limits; `Guaranteed` for databases and whatever must not be evicted.
- **One namespace per application and per environment**, with LimitRange and ResourceQuota set by the platform; developers get `edit` on it, `view` elsewhere.
- **Applications have a dedicated ServiceAccount**, with no right by default; operators and agents (CI, GitOps) have precise, audited roles.
- **Deny-all + explicit allowances** in NetworkPolicy, at least in production; the database accepts only the API.
- **Observability** is a platform service (Prometheus, Loki, Grafana) reading what your Pods expose: `stdout` , `/actuator/prometheus` , the Events.

Remember

- `requests` reserve and place, `limits` cap; memory exceeded = `OOMKilled` (137), CPU exceeded = throttled; impossible request = `Pending` .
- QoS `Guaranteed` > `Burstable` > `BestEffort` : the order in which the node sacrifices Pods.
- Namespace = partition; LimitRange = per-container defaults and ceilings; ResourceQuota = the namespace's budget.
- `top` , `events` , `describe` , `logs --previous` , `debug --target` : the diagnostic kit, without adding anything to the images.
- RBAC = Role (what) + RoleBinding (who); `auth can-i --as=` to test; an application has a ServiceAccount with no rights.

- NetworkPolicy = allow-list by label, enforced by the CNI (k3s does); without a policy, everything talks to everything.

Vocabulary

request / limit: reservation for the scheduler / cgroup ceiling. — **QoS**: `Guaranteed`, `Burstable`, `BestEffort`. — **`OOMKilled`**: container killed for exceeding memory (137). — **throttling**: CPU slow-down at the limit. — **LimitRange**: per-container defaults and ceilings in a namespace. — **ResourceQuota**: total budget of a namespace. — **metrics-server**: source of `kubectl top`. — **ephemeral container**: diagnostic container added by `kubectl debug`. — **RBAC**: Role, ClusterRole, RoleBinding, ClusterRoleBinding. — **ServiceAccount**: a Pod's identity towards the API. — **`kubectl auth can-i`**: permission test. — **NetworkPolicy**: traffic rules between Pods, enforced by the CNI.